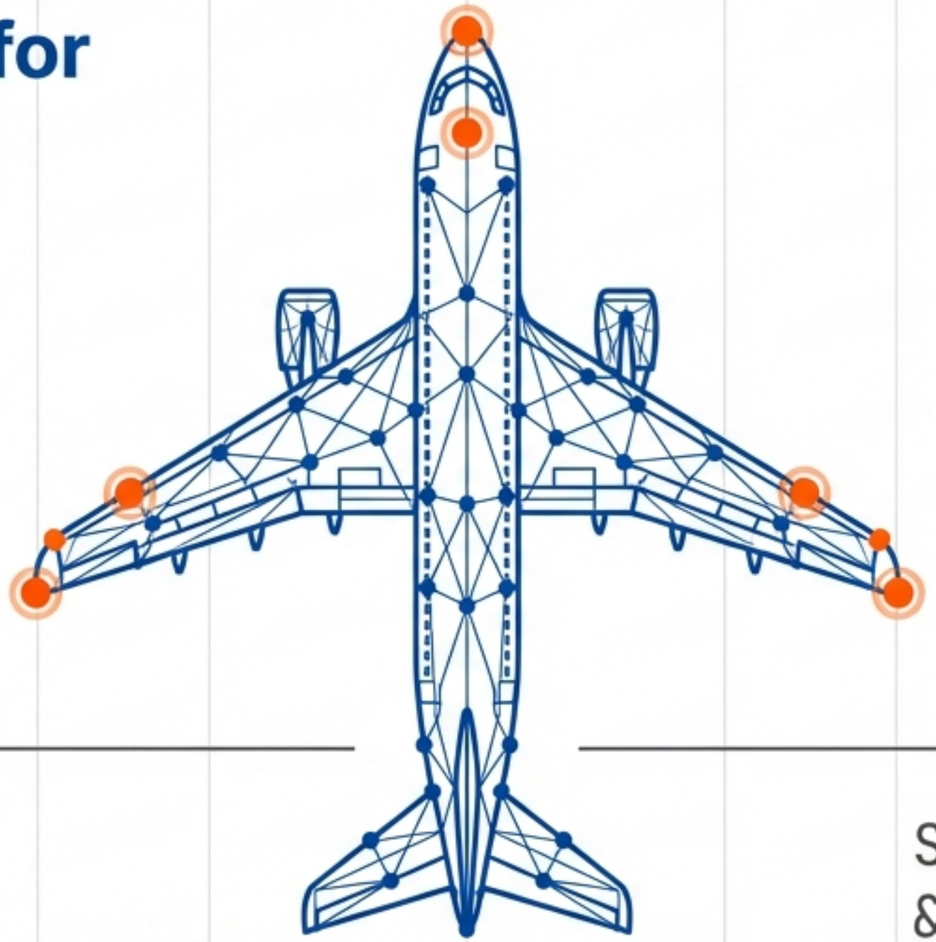
# AeroDyn Model Factory

## A Flight Simulator for Business Strategy

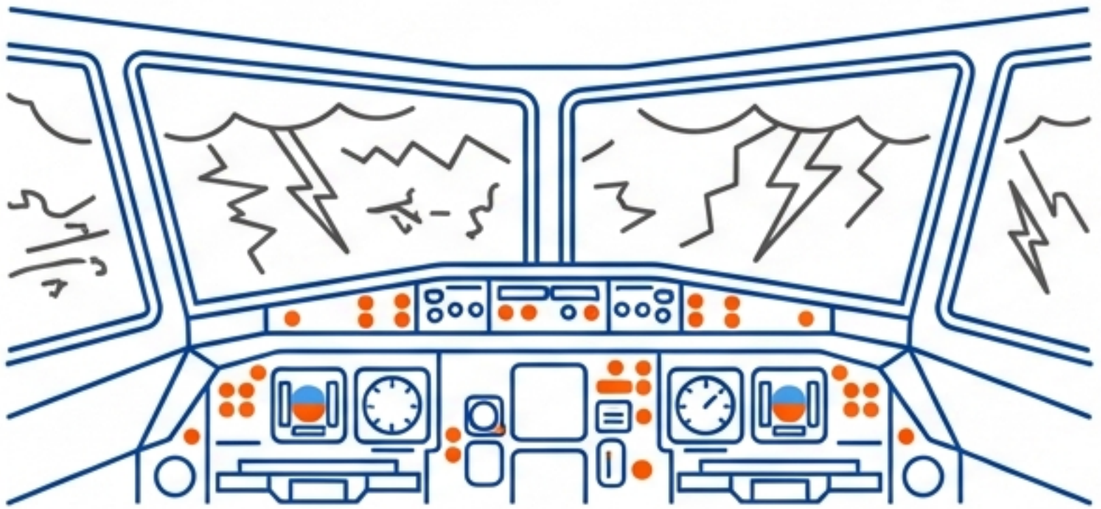


System Documentation
& Conceptual Overview

Architecture: System Dynamics + JSON + AI Orchestration

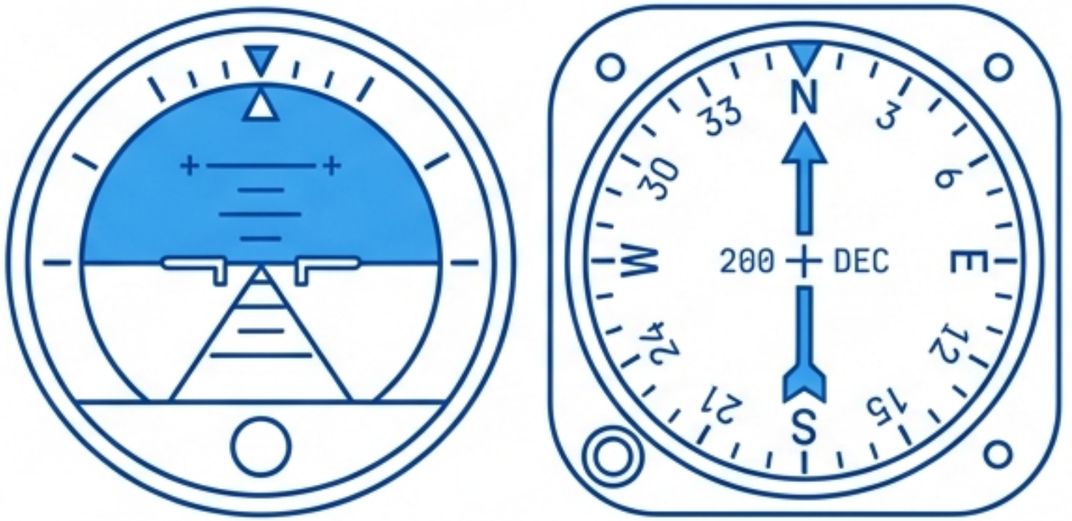
# You Would Never Pilot an Aircraft Based on Guesswork

## THE REAL WORLD: High Risk.



Testing a radical pricing change or ethical shift with real passengers (employees) is dangerous.

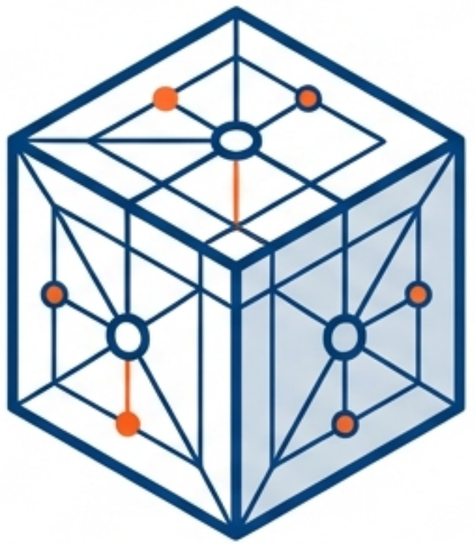
## THE SIMULATOR: Safe Sandbox.




AeroDyn allows leadership to crash a digital company a thousand times to ensure the real one lands safely.

**Key Insight:** This is not just a spreadsheet; it is a sandbox for reality.

# Three Objectives of the Simulation Engine

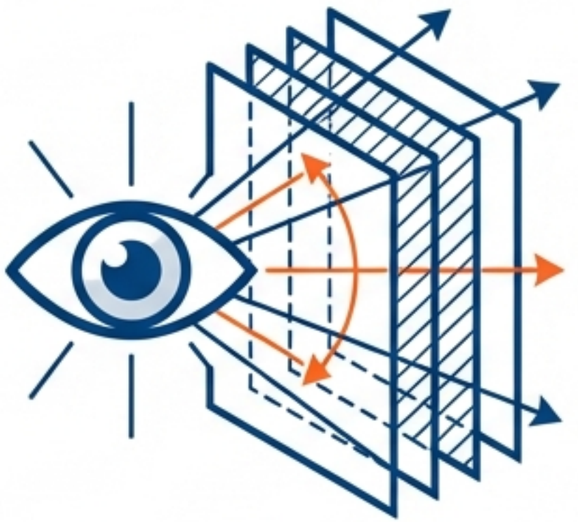


## 1. Model Reality

Create a precise virtual copy of the enterprise. Capture intangible assets like reputation alongside tangible ones like contracts and capacity.

## 2. Test the Future

Project consequences over a 5+ year timeline. Answer the critical question: 'Does this strategy lead to success or bankruptcy?'



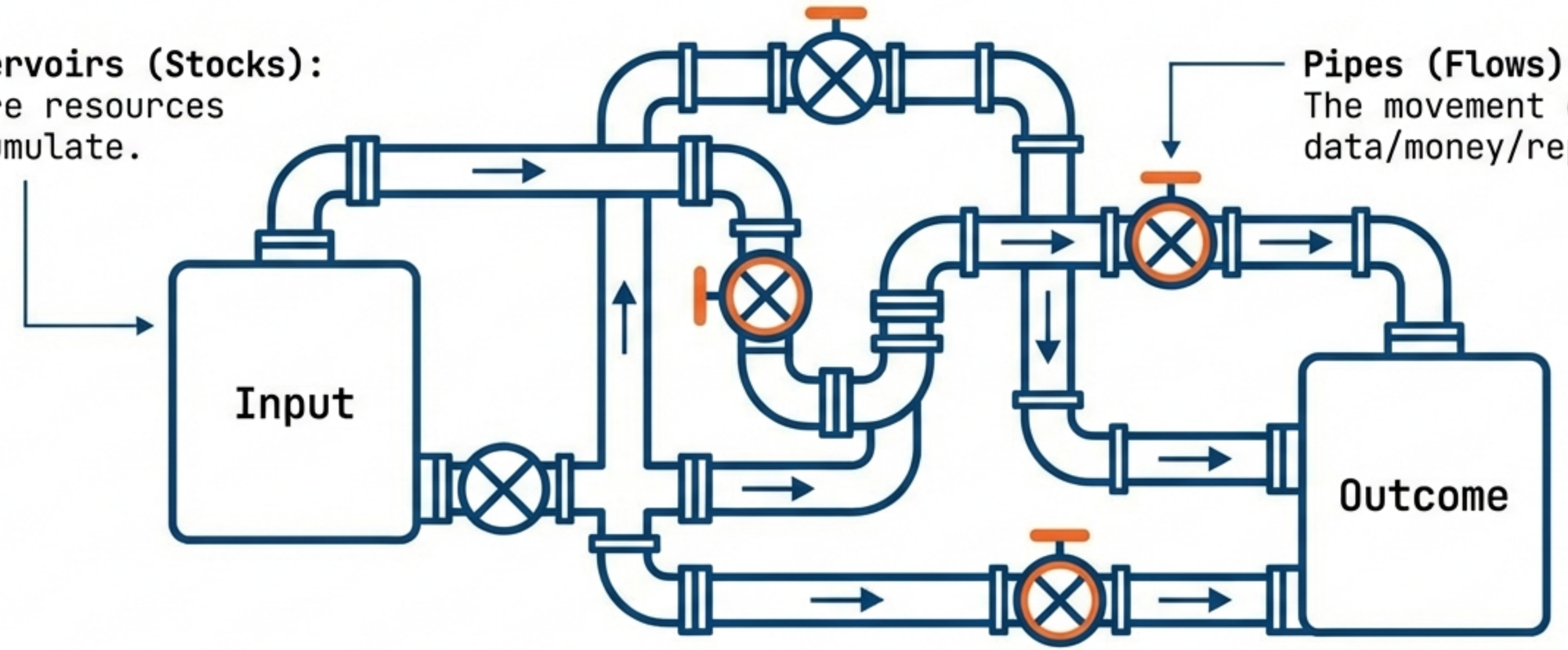
## 3. See the Invisible

Uncover feedback loops and side effects. Reveal how a short-term win might trigger a long-term collapse.

# Under the Hood: The ‘Hydraulics’ of Business

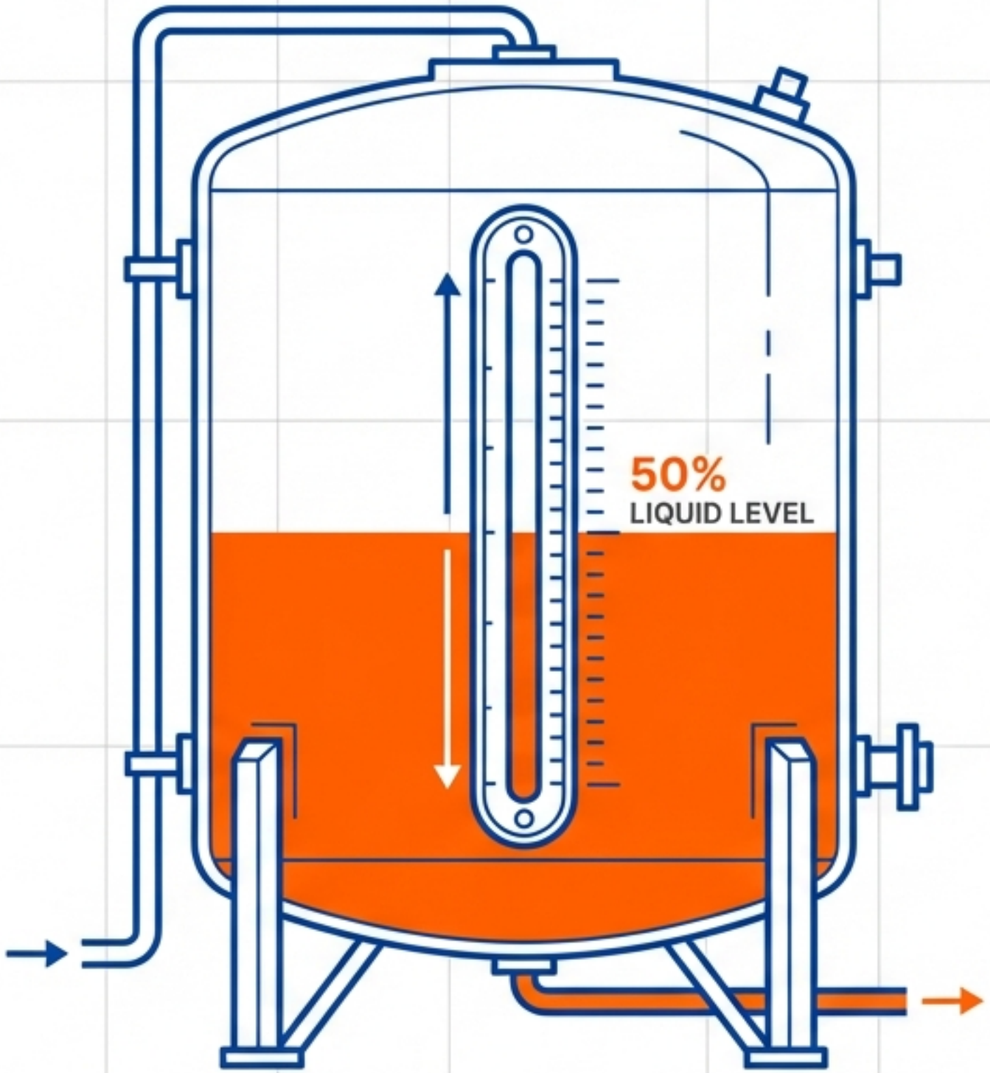
**Reservoirs (Stocks):**
Where resources accumulate.

**Pipes (Flows):**
The movement of data/money/reputation.




The system relies on System Dynamics. The structure is defined in a simple text file: 'model.json'. Understanding the flow of 'liquid' through these pipes reveals the true health of the organization.

# Component A: Stocks (The Reservoirs)




## Definition

Stocks represent the state of the system at a specific moment (Time T). They are the accumulations of history.

## Analogy

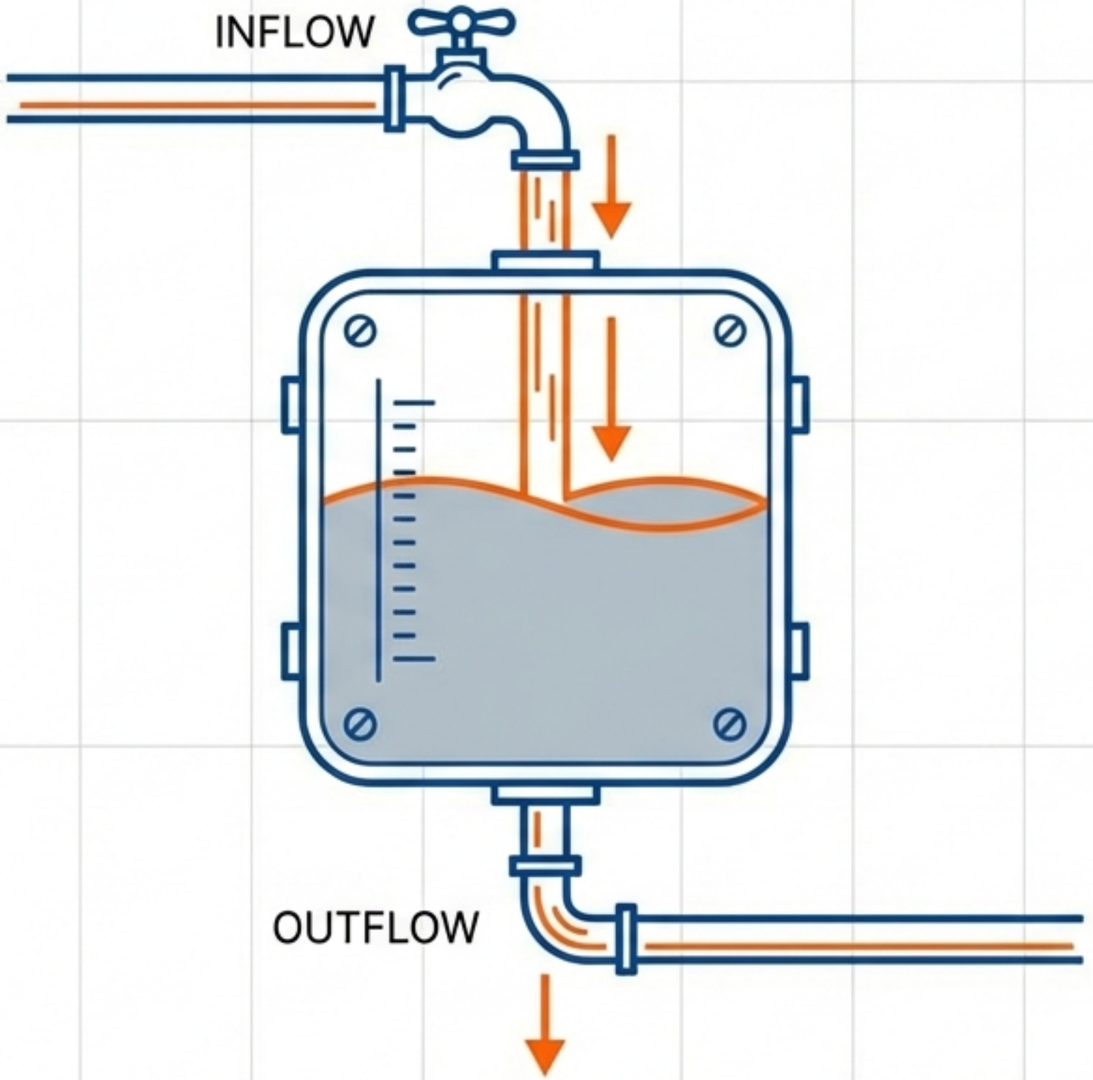
Think of water sitting in a bathtub. It is static unless a force acts upon it.

## Code Examples (in JetBrains Mono)

`reputation_capital` Current level of trust

`ai_investment_fund` Cash in bank

# Component B: Flows (The Pipes)




## Definition

Flows are the forces of change. They determine if Stocks rise or fall.

## The Math

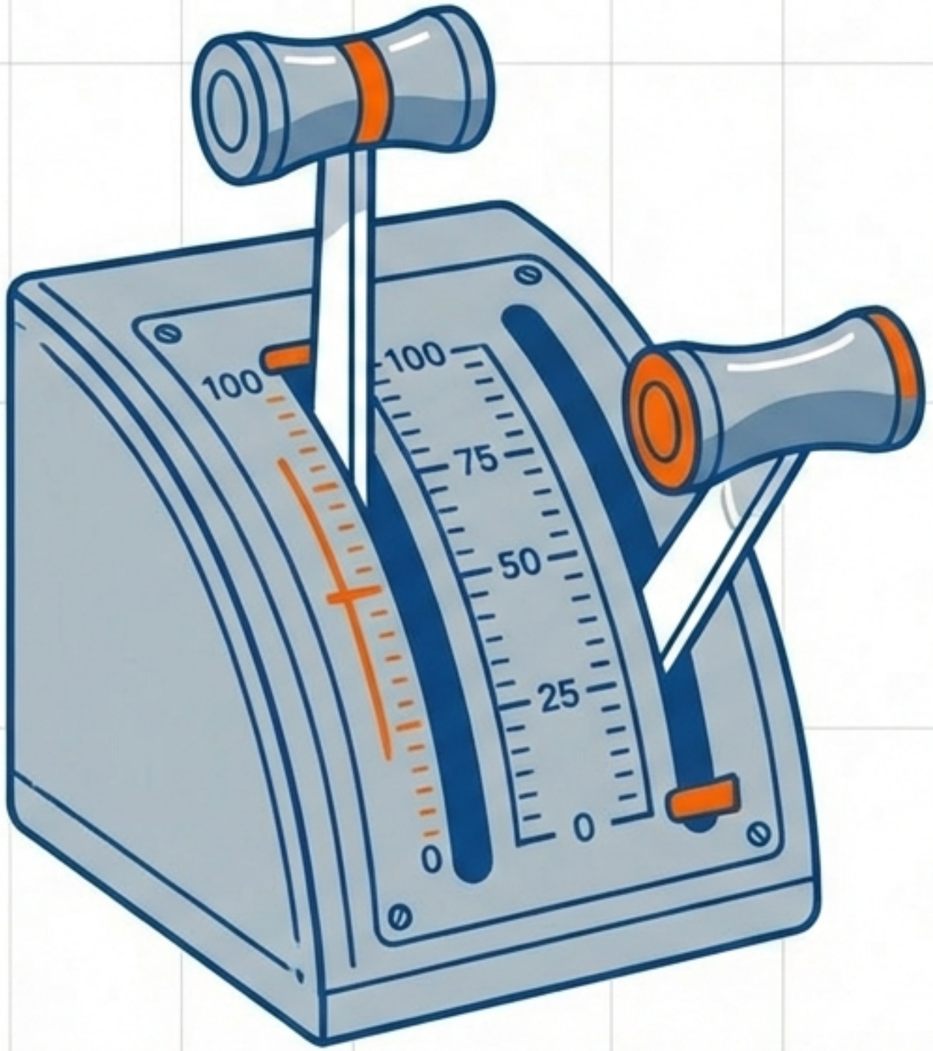
Every pipe is governed by a specific formula. E.g., "Public fear grows in proportion to weapon sales."

## Code Examples (in JetBrains Mono)

`capacity_expansion` (Inflow: Building new factories)

`reputation_decay` (Outflow: Public forgetting)

# Component C: Parameters (The Levers)




## Definition

Parameters are static inputs controlled by human decision-makers. These are the variables you can touch.

## Analogy

The knobs, switches, and levers in the cockpit.

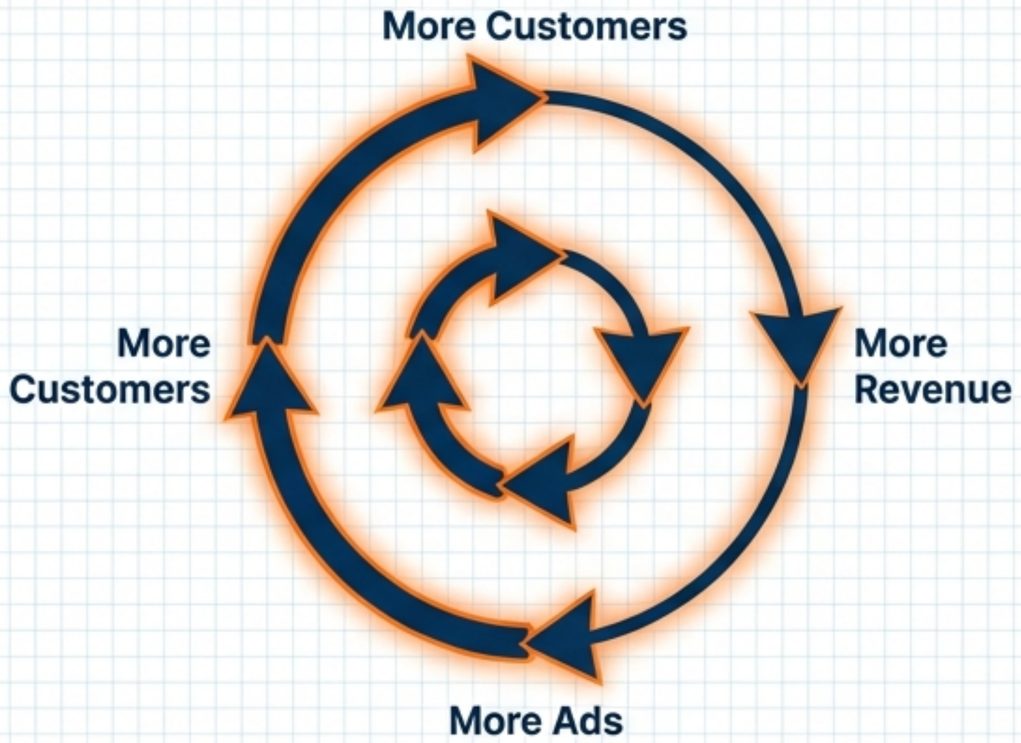
## Code Examples (in JetBrains Mono)

`ai_investment_rate` (R&D Budget Allocation %)

`ethical_threshold` (Risk Tolerance Setting)

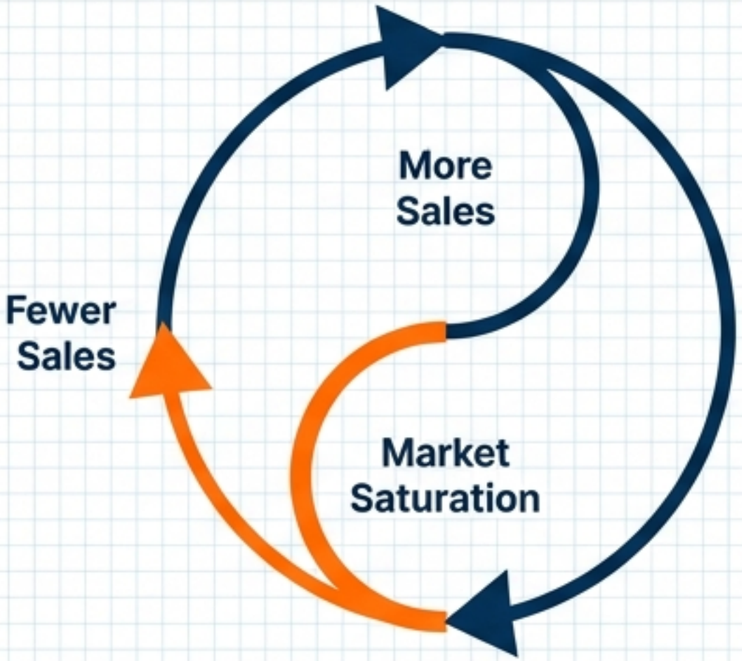
# Component D: The Magic of Feedback Loops

## Reinforcing Loop (Snowball)




Actions that compound. More Customers -> More Revenue -> More Ads -> More Customers.

## Balancing Loop (Thermostat)




Actions that self-correct. More Sales -> Market Saturation -> Fewer Sales.

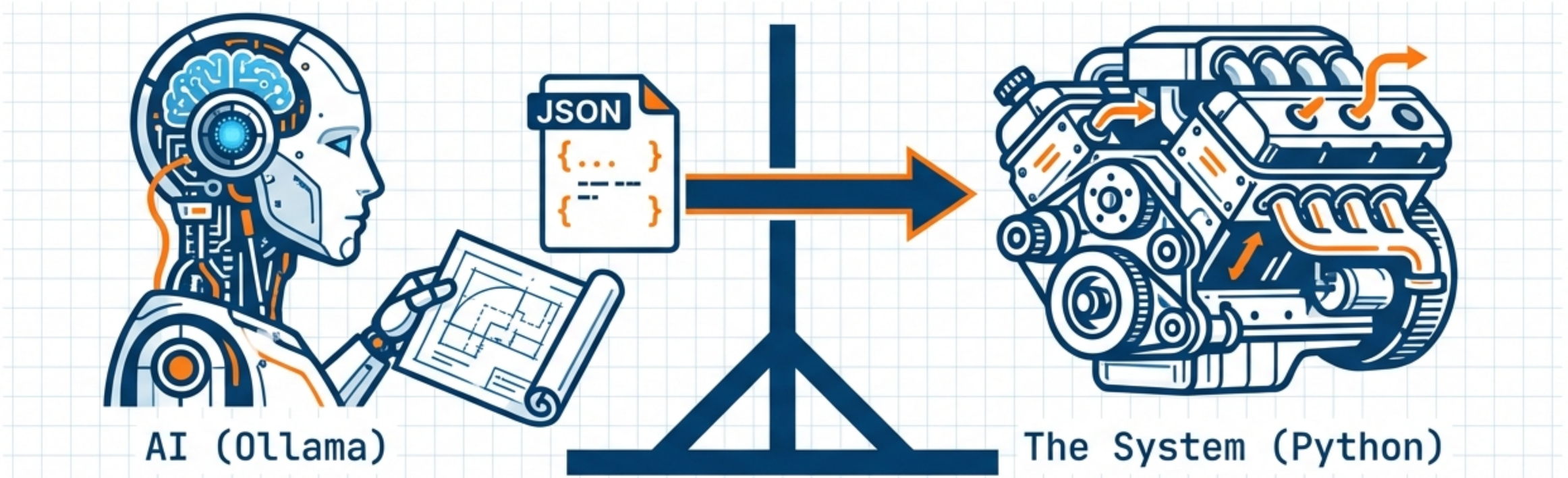
# The Blueprint: Defined in JSON

```
{
  "stock": "Reputation",
  "flow": "Decay",
  "rate": 0.05
}
```

**model.json**


The structure *is* the logic. Changing the text file changes the reality of the simulation.

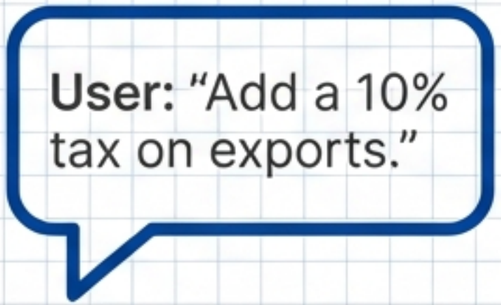
# The Co-Pilot: AI as the Architect




- Role: The AI bridges the gap between natural language and system structure.
- Constraint: The AI does NOT touch the Python code. It only reads and edits the Blueprint (model.json).
- Function: It reads the current state and suggests renovations.

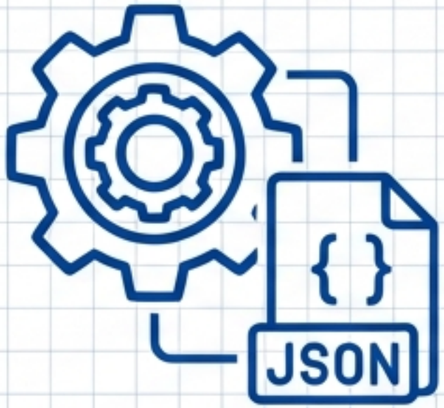
# From Intent to Execution: The Interaction Flow

## 1. The Command




**User:** “Add a 10% tax on exports.”

## 2. The Translation




**AI:** Creates flow “tax_export”.
Formula: “Revenus * 0.10”.

## 3. The Build


**System:** Installs the pipe. Simulation updates immediately.

# The Philosophy: Why "No-Code"?

## Zero Code Required

Strategy changes shouldn't require a developer. Modify data (JSON), not application code.

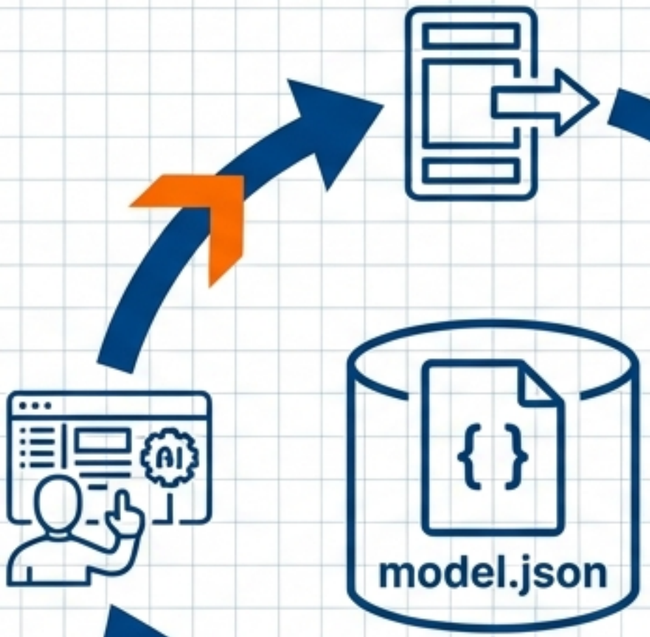
## Total Transparency

No “Black Box” algorithms. Every formula is visible in the JSON file.


## Auditability

Trace exactly why a crash happened. E.g., “Reputation tanked because Ethical Threshold was too low.”

# System Architecture & Data Cycle




**READ**
Parse model.json

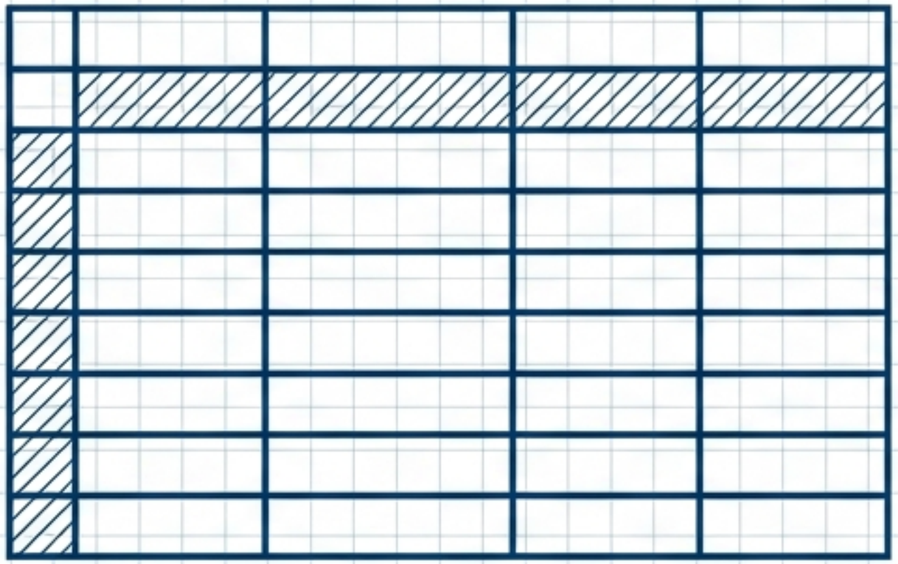
**SIMULATE**
Calculate Time Steps

**DISPLAY**
Render Interface

**MODIFY**
User/AI Interaction

# More Than a Spreadsheet: A Living Machine

## Traditional Tools



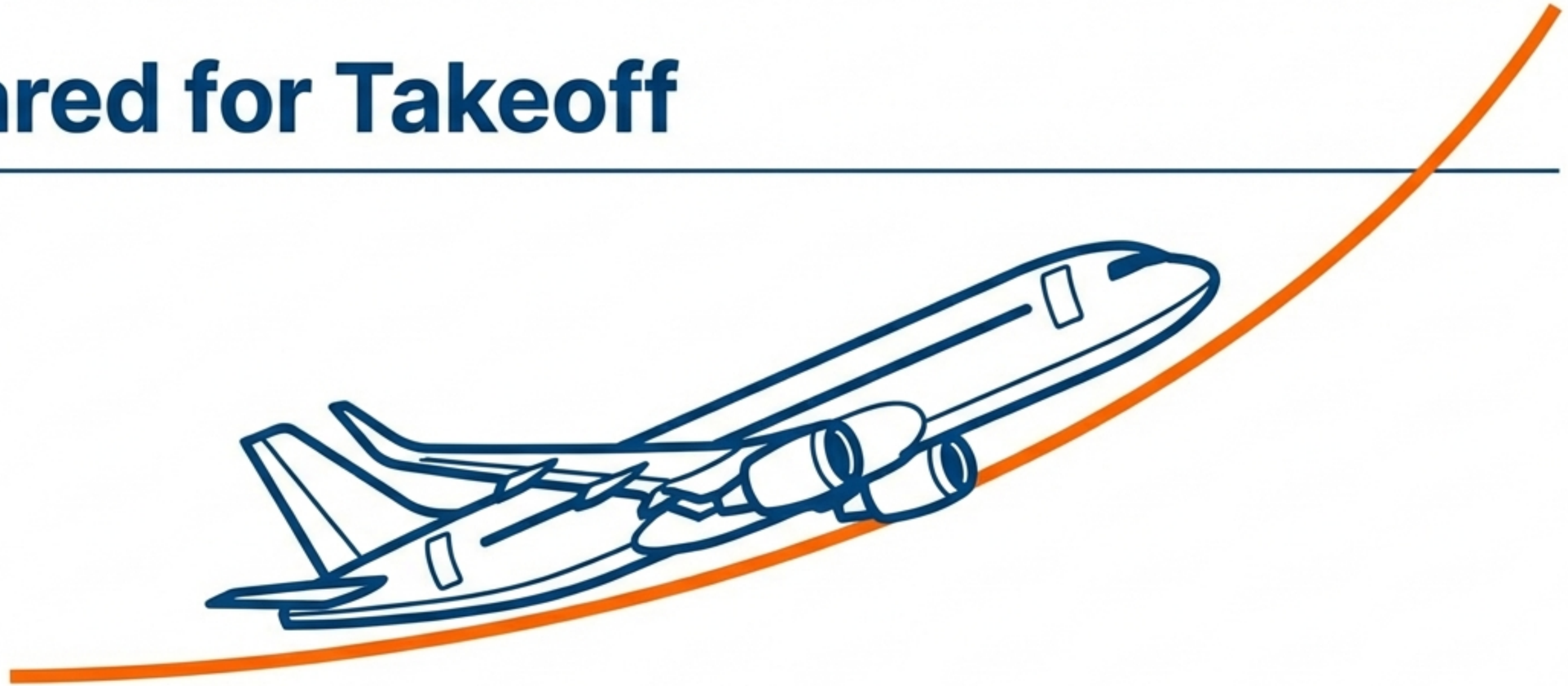
A Snapshot. A static picture of the world as it is.

## AeroDyn

A Movie. A dynamic simulation of how the world evolves.

A logic machine designed to reveal consequences before it is too late.

# Cleared for Takeoff



Complexity shouldn't be feared; it should be modeled.
Treat strategy as engineering, not intuition.

## Ready to pilot your strategy?